

Tutorato 5

Sara Trabucco

20 Maggio 2026

- Caricare il dataset `marketing_campaign.csv`; poi:

```
data <- read.csv("marketing_campaign.csv", sep = "\t")

str(data)
head(data)
summary(data)
```

Preprocessing del dataset

- Classificare le principali variabili del dataset; che variabili potrebbero essere più adatte ad una cluster analysis (considerando che l'obiettivo è una *customer personality analysis*, utile per le aziende per comprendere la propria clientela)?

Per la cluster analysis conviene usare variabili che descrivono il profilo e il comportamento del cliente (ad esempio, quanto compra e quanto spende, quanto frequenta il sito, qual è la disponibilità economica, l'età, etc.)

L'identificativo ID va escluso a priori perché non contiene informazioni utili (serve solo per identificare univocamente i clienti).

- Verificare la presenza di valori mancanti (dove compaiono, quanti sono, etc.) e rimuoverli dal dataset;

```
colSums(is.na(data))
```

```
data <- na.omit(data)
```

In questo caso rimuoviamo le osservazioni incomplete per semplicità (spesso si cerca di capire anche di che tipo siano i valori mancanti).

- Analizzare le variabili qualitative, trasformarle in factor e ricodificare i livelli;

```
unique(data$Education) # per verificare se ci sono eventuali typos
unique(data$Marital_Status)

table(data$Education)
table(data$Marital_Status)

data$Education <- factor(
  data$Education,
  levels = c("Basic", "2n Cycle", "Graduation", "Master", "PhD"),
  ordered = TRUE
)

data$Marital_Status <- as.character(data$Marital_Status)
```

```
data$Marital_Status[
  data$Marital_Status == "Alone" |
  data$Marital_Status == "YOLO" |
  data$Marital_Status == "Absurd"
] <- "Single"

unique(data$Education)
unique(data$Marital_Status)
data$Marital_Status <- factor(data$Marital_Status)

table(data$Education)
table(data$Marital_Status)
```

- Creare le variabili:
 - Age (età del cliente)
 - Total_spending (spesa totale su tutti i prodotti)
 - Total_purchases (numero totale degli acquisti)

```
data$Age <- 2026 - data$Year_Birth

data$Total_spending <- data$MntWines +
  data$MntFruits +
  data$MntMeatProducts +
  data$MntFishProducts +
  data$MntSweetProducts +
  data$MntGoldProds

data$Total_purchases <- data$NumWebPurchases +
  data$NumCatalogPurchases +
  data$NumStorePurchases
```

Age, Total_spending e Total_purchases sintetizzano informazioni sparse nel dataset e rendono più interpretabili i profili dei clienti (pro: riducono il numero di variabili, contro: perdiamo un po' di informazioni specifiche)

```
cluster_data <- data[, c(
  "Income",
  "Age",
  "Total_spending",
  "Total_purchases",
  "NumWebVisitsMonth"
)]

summary(cluster_data)
```

- Standardizzare (ove opportuno) le variabili;

```
cluster_scaled <- scale(cluster_data)
```

La standardizzazione è necessaria perché le variabili hanno scale diverse. Senza standardizzazione, variabili come Income o Total_spending dominerebbero il calcolo delle distanze (vista la loro scala rispetto, ad esempio, Age).

In particolare ogni variabile è stata normalizzata sottraendo la media e dividendo per la deviazione standard (quindi ora ha media 0 e sd pari a 1).

Nell'oggetto `cluster_scaled` sono anche salvate informazioni riguardanti le medie e le sd (negli attributi).

Analisi esplorativa

- Considerare le variabili `Income`, `Age`, `Total_spending`, `Total_purchases`, `NumWebVisitsMonth` e calcolare indici di posizione, indici di dispersione, rappresentarne graficamente la distribuzione e analizzare eventuali outlier;

```
summary(cluster_data)
```

```
sapply(cluster_data, mean)
sapply(cluster_data, median)
sapply(cluster_data, sd)
sapply(cluster_data, IQR)
```

```
library(e1071)
```

```
sapply(cluster_data, skewness)
```

```
par(mfrow = c(2, 3))

for (var in names(cluster_data)) {
  hist(cluster_data[[var]],
        main = var,
        xlab = var,
        col = "steelblue",
        border = "white")
}

par(mfrow = c(1, 1))
```

```
par(mfrow = c(2, 3))

for (var in names(cluster_data)) {
  boxplot(cluster_data[[var]],
           main = var,
           col = "steelblue")
}

par(mfrow = c(1, 1))
```

I boxplot permettono di individuare clienti con valori estremi, ad esempio reddito o spesa molto elevati; questi clienti possono influenzare molto k-means ad esempio

- Costruire scatterplot e matrice di correlazione tra le principali variabili quantitative; che relazioni emergono?

```
pairs(cluster_data,
       pch = 19,
       col = "steelblue")
```

```
round(cor(cluster_data), 2)
```

```
library(GGally)
ggpairs(cluster_data)
```

- Calcolare la matrice delle distanza utilizzando la distanza euclidea e distanza di Manhattan;

```
dist_euclidean <- dist(cluster_scaled, method = "euclidean")
dist_manhattan <- dist(cluster_scaled, method = "manhattan")
```

La distanza euclidea misura la distanza “in linea d’aria” tra osservazioni, mentre la distanza Manhattan somma le differenze assolute sulle singole variabili. Per entrambe lavoriamo su variabili quantitative e **standardizzate** (!)

- Che misura di *dissimilarità* è appropriato usare con variabili **qualitative**?

```
library(cluster)

mixed_data <- data[, c(
  "Income",
  "Age",
  "Total_spending",
  "Total_purchases",
  "NumWebVisitsMonth",
  "Education",
  "Marital_Status"
)]

gower_dist <- daisy(mixed_data, metric = "gower")
```

Per dati misti, cioè con variabili quantitative e qualitative, una misura appropriata è la **distanza di Gower**, perché combina informazioni numeriche e categoriali

Clustering gerarchico

Il clustering gerarchico parte da n cluster, uno per osservazione, poi passo dopo passo:

1. cerca i due cluster più vicini
2. li fonde
3. ricalcola le distanze
4. ripete fino ad avere un unico cluster

Come definire la distanza tra cluster?

- Applicare clustering gerarchico utilizzando *single linkage*, *complete linkage*, *average linkage* e *Ward*;

Single linkage: la distanza tra due cluster è $d(A, B) = \min_{i \in A, j \in B} d(i, j)$; tra tutti i punti dei due cluster, prendo la distanza minima:

```
hc_single <- hclust(dist_euclidean, method = "single")

plot(hc_single,
     labels = FALSE,
     main = "Clustering gerarchico - Single linkage",
     xlab = "",
     sub = "")
```

Per visualizzare meglio il dendrogramma lavoriamo con un subset del dataset originale:

```
set.seed(123)

idx <- sample(1:nrow(cluster_scaled), 100)
cluster_subset <- cluster_scaled[idx, ]
dist_subset <- dist(cluster_subset)
```

```
hc_single <- hclust(dist_subset,
                   method = "single")

plot(hc_single,
     labels=FALSE,
     main="Single linkage")
```

Il single linkage può produrre il cosiddetto **chaining effect**: osservazioni collegate progressivamente in catene lunghe, con cluster poco compatti.

Complete linkage: tra tutte le distanze possibili tra punti dei cluster, calcolo tutte le distanze e prendo la massima, quindi $d(A, B) = \max_{i \in A, j \in B} d(i, j)$

```
hc_complete <- hclust(dist_euclidean, method = "complete")

plot(hc_complete,
     labels = FALSE,
     main = "Clustering gerarchico - Complete linkage",
     xlab = "",
     sub = "")
```

Il complete linkage tende a produrre cluster più compatti rispetto al single linkage.

```
set.seed(123)

idx <- sample(1:nrow(cluster_scaled), 100)
cluster_subset <- cluster_scaled[idx, ]
dist_subset <- dist(cluster_subset)
hc_complete <- hclust(dist_subset,
                    method = "complete")

plot(hc_complete,
     labels=FALSE,
     main="Complete linkage")
```

Average linkage: è un compromesso tra single e complete, infatti tra tutti i punti dei due cluster si calcolano tutte le distanze e si fa la media $d(A, B) = \frac{1}{|A||B|} \sum_{i \in A, j \in B} d(i, j)$.

```
hc_average <- hclust(dist_euclidean, method = "average")

plot(hc_average,
     labels = FALSE,
     main = "Clustering gerarchico - Average linkage",
     xlab = "",
     sub = "")
```

```
set.seed(123)

idx <- sample(1:nrow(cluster_scaled), 100)
cluster_subset <- cluster_scaled[idx, ]
dist_subset <- dist(cluster_subset)
hc_average <- hclust(dist_subset,
                    method = "average")

plot(hc_average,
     labels=FALSE,
```

```
main="Ward")
```

Metodo di Ward: l'idea è chiedersi se, unendo due cluster, l'omogeneità interna peggiori; vogliamo avere cluster compatti e con bassa variabilità interna. Per farlo calcoliamo la **devianza interna**:

$$DEV(C) = \sum_{i \in C} (x_i - \bar{x}_C)^2$$

e ad ogni passo, scegliamo la fusione che minimizza ΔDEV

```
hc_ward <- hclust(dist_euclidean, method = "ward.D2")  
# perché implementa correttamente la variazione della devianza
```

Vediamo due salti:

- da 44 a 97
- da 35 a 44

Il grafico delle altezze ordinate delle fusioni mostra alcuni salti particolarmente marcati, in particolare nelle ultime fusioni. Ciò suggerisce che l'algoritmo, per ridurre ulteriormente il numero di cluster, dovrebbe unire gruppi molto diversi tra loro

```
plot(hc_ward,  
     labels = FALSE,  
     main = "Clustering gerarchico - Ward",  
     xlab = "",  
     sub = "")
```

```
set.seed(123)  
  
idx <- sample(1:nrow(cluster_scaled), 100)  
cluster_subset <- cluster_scaled[idx, ]  
dist_subset <- dist(cluster_subset)  
hc_ward_sub <- hclust(dist_subset,  
                     method = "ward.D2")  
  
plot(hc_ward_sub,  
     labels=FALSE,  
     main="Ward")
```

- Utilizzando il metodo di Ward, scegliere un numero appropriato di cluster;

```
plot(sort(hc_ward$height,  
         decreasing=TRUE),  
     type="b",  
     pch=19,  
     main="Salti nelle altezze")
```

Ha senso tagliare prima di un grande salto, perché un grande salto implica la fusione di due gruppi molto diversi

Dopo aver costruito la gerarchia, otteniamo la partizione finale in 2 gruppi:

```
ward_clusters <- cutree(hc_ward, k = 2)  
clusters <- cutree(hc_ward,  
                  h = 45)
```

```
table(ward_clusters)
table(clusters)
```

```
plot(hc_ward_sub,
     labels = FALSE,
     main = "Clustering gerarchico - Ward",
     xlab = "",
     sub = "")

rect.hclust(hc_ward_sub, k = 2, border = c(3,4))
```

Se volessimo colorare i *rami* del dendrogramma:

```
library(dendextend)
dend <- as.dendrogram(hc_ward_sub)

dend <- color_branches(dend,
                       k=2)

plot(dend)
```

Per analizzare le variabili rispetto alla loro appartenenza ai cluster:

```
aggregate(cluster_data,
           by = list(Cluster = ward_clusters),
           FUN = mean)
```

```
data$ward_cluster <- factor(ward_clusters)

table(data$ward_cluster, data$Education)
table(data$ward_cluster, data$Marital_Status)
```

```
par(mfrow = c(2, 3))

for (var in names(cluster_data)) {
  boxplot(cluster_data[[var]] ~ data$ward_cluster,
          main = var,
          xlab = "Cluster Ward",
          ylab = var,
          col = c("steelblue", "coral"))
}

par(mfrow = c(1, 1))
```

In questo caso:

- il reddito non sembra essere la variabile principale che contribuisce alla separazione dei cluster e neanche l'età
- la spesa totale e gli acquisti totali invece contribuiscono fortemente alla segmentazione
- sembra che i clienti del cluster 2 visitino frequentemente ma acquistino poco

Sembra che il cluster 1 sia composto da clienti che comprano spesso, spendono molto e probabilmente già sanno cosa acquistare (in gergo, *high-value customers*) mentre quelli del cluster 2 siano clienti poco coinvolti o potenziali (o *browsers* o *lurkers*).

K-means

- Applicare l'algoritmo K-means ai dati standardizzati; determinare un numero appropriato di cluster considerando:
 - il *metodo del gomito* (Elbow Method);
 - il *silhouette score*;

In K-means si minimizza la **within cluster sum of squares**, che misura quanto i punti sono dispersi all'interno dei cluster; se aumentiamo K, la WCSS può solo diminuire o restare uguale.

L'interpretazione è:

- se il gomito è a K=3, significa che oltre 3 cluster si ottiene un miglioramento ridotto

```
set.seed(123)

wss <- sapply(1:10, function(k) {
  kmeans(cluster_scaled, centers = k, nstart = 25)$tot.withinss
})

plot(1:10, wss,
     type = "b",
     pch = 19,
     col = "steelblue",
     xlab = "Numero di cluster K",
     ylab = "Within-cluster sum of squares",
     main = "Metodo del gomito")
```

Il metodo del gomito suggerisce che il numero ottimale di cluster sia compreso tra 2 e 3, poiché oltre questi valori la riduzione della variabilità interna ai cluster diventa molto meno marcata. In particolare, K = 3 rappresenta spesso un buon compromesso tra semplicità interpretativa e capacità descrittiva (perché stiamo analizzando il comportamento di clienti, avere troppi “tipi” di clienti potrebbe essere difficile da interpretare)

Per il **silhouette score** non si guarda solo alla compattezza interna ma anche quanto i cluster siano separati tra loro; per un'osservazione i se il silhouette score $s(i)$ risulta essere superiore a 0.5, significa che è stata ben assegnata ai gruppi.

```
library(cluster)

sil_values <- numeric(9)

for (k in 2:10) {
  km_temp <- kmeans(cluster_scaled, centers = k, nstart = 25)
  sil <- silhouette(km_temp$cluster, dist_euclidean)
  sil_values[k - 1] <- mean(sil[, 3])
}

plot(2:10, sil_values,
     type = "b",
     pch = 19,
     col = "steelblue",
     xlab = "Numero di cluster K",
     ylab = "Silhouette media",
     main = "Silhouette score")
```

Il silhouette score raggiunge il massimo per K = 2, suggerendo che la partizione in due cluster sia quella con la migliore separazione media tra gruppi. Tuttavia, valori di K maggiori (ad esempio 3) potrebbero

risultare comunque utili se si desidera una segmentazione più dettagliata (pur rinunciando ad una “migliore” separazione)

```
km2 <- kmeans(cluster_scaled, centers = 2, nstart = 25)
km3 <- kmeans(cluster_scaled, centers = 3, nstart = 25)

aggregate(cluster_data, by = list(Cluster = km2$cluster), mean)

aggregate(cluster_data, by = list(Cluster = km3$cluster), mean)
```

- Una volta scelto K:
 - rappresentare graficamente i cluster;
 - analizzare i centroidi;

```
set.seed(123)

km <- kmeans(cluster_scaled,
             centers = 2,
             nstart = 25)

# dimensione dei cluster
km$size
table(km$cluster)

plot(cluster_data$Income,
     cluster_data$Total_spending,
     col = km$cluster,
     pch = 19,
     xlab = "Income",
     ylab = "Total spending",
     main = "K-means clustering (K = 2)")

legend("topright",
     legend = paste("Cluster", 1:2),
     col = 1:2,
     pch = 19)
```

Per vedere i centroidi nella forma originale, sfruttiamo che quando facciamo `scale()`, R restituisce la matrice standardizzata e salva anche la informazioni usate per la trasformazione:

```
attributes(cluster_scaled)

km$centers

centroidi_originali <-
km$centers *
attr(cluster_scaled, "scaled:scale") +
attr(cluster_scaled, "scaled:center")

centroidi_originali

par(mfrow = c(2, 3))

for (var in names(cluster_data)) {
  boxplot(cluster_data[[var]] ~ km$cluster,
```

```

    main = paste(var, "by cluster"),
    xlab = "Cluster",
    ylab = var,
    col = c("steelblue", "coral"))
}

par(mfrow = c(1, 1))

```

PAM

- Applicare PAM agli stessi dati; confrontare poi PAM e K-means rispetto alla robustezza agli outlier, interpretabilità e struttura dei cluster;

PAM usa **medoidi** (osservazioni reali) invece di minimizzare il WCSS, quindi il metodo del gomito è meno naturale; con PAM usiamo quasi sempre **la silhouette**

```

library(cluster)

pam_fit <- pam(cluster_scaled, k = 2)
str(pam_fit)

table(pam_fit$clustering)

```

Per visualizzare i centroidi:

```

pam_fit$id.med
data[pam_fit$id.med, ]

```

```

aggregate(cluster_data,
  by = list(Cluster = pam_fit$clustering),
  FUN = mean)

```

PAM usa medoidi, cioè osservazioni reali rappresentative del cluster e per questo è più robusto agli outlier rispetto a K-means. Possiamo osservare la silhouette media:

```

barplot(
  pam_fit$silinfo$clus.avg.widths,
  col = c("steelblue", "coral"),
  ylab = "Silhouette media",
  xlab = "Cluster",
  main = "Silhouette media per cluster"
)

# silhouette media totale
abline(h = pam_fit$silinfo$avg.width,
  col = "red",
  lwd = 2,
  lty = 2)

```

O prendere un subset di osservazioni:

```

set.seed(123)

idx <- sample(1:nrow(cluster_scaled), 200)

pam_small <- pam(cluster_scaled[idx, ], k=2)

```

```
plot(
  silhouette(pam_small),
  main="Silhouette plot (campione)"
)
```

E per scegliere K possiamo basarci sul silhouette score:

```
library(cluster)

sil_width <- numeric(9)

for(k in 2:10){

  pam_fit1 <- pam(cluster_scaled,
                  k=k)

  sil_width[k-1] <- pam_fit1$silinfo$avg.width
}

plot(2:10,
     sil_width,
     type="b",
     pch=19,
     col="steelblue",
     xlab="Numero di cluster",
     ylab="Silhouette media",
     main="Scelta di K con PAM")
```

```
mean(silhouette(pam_fit)[, 3])
```

Le osservazioni finiscono negli stessi gruppi nei due algoritmi?

```
table(Kmeans = km$cluster,
      PAM = pam_fit$clustering)
```

I due algoritmi hanno trovato praticamente la stessa partizione, solo invertita. Solo 30 osservazioni su 2216 vengono classificate diversamente (1.35%).

- Applicare PAM utilizzando una matrice di dissimilarità basata su Gower e confrontare i risultati con PAM ottenuto al punto precedente;

```
mixed_data <- data[, c(
  "Income",
  "Age",
  "Total_spending",
  "Total_purchases",
  "NumWebVisitsMonth",
  "Education",
  "Marital_Status"
)]

# education e marital status sono qualitative

gower_dist <- daisy(mixed_data, metric = "gower")
```

```
pam_gower <- pam(gower_dist, k = 2, diss = TRUE)

table(pam_gower$clustering)
```

Di nuovo, analizziamo le caratteristiche dei clienti dei vari cluster:

```
aggregate(cluster_data,
           by = list(Cluster = pam_gower$clustering),
           FUN = mean)
```

```
table(PAM_standard = pam_fit$clustering,
      PAM_Gower = pam_gower$clustering)
```

PAM con Gower permette di includere anche variabili qualitative, come `Education` e `Marital_Status`; ci aspettiamo che i cluster possano cambiare perché la nozione di somiglianza include più aspetti del profilo cliente:

```
par(mfrow = c(2,3))

for (var in names(cluster_data)) {

  boxplot(
    cluster_data[[var]] ~ pam_gower$clustering,
    main = paste(var, "by cluster"),
    xlab = "Cluster",
    ylab = var,
    col = c("steelblue", "coral")
  )
}

par(mfrow = c(1,1))
```

DBSCAN

- Applicare DBSCAN scegliendo *opportunamente* `eps` e `minPts`; identificare cluster e rumore e identificare graficamente i risultati.

DBSCAN cerca zone dense di punti e non richiede di specificare `K`, trova forme arbitrarie e identifica outliers automaticamente. Punti vicini = cluster, punti isolati = rumore.

`eps` è il raggio del vicinato, `minPts` è il numero minimo di punti richiesti.

DBSCAN permette di identificare tra tipi di punti: i core points, i border points ed i noise points.

Di solito $\text{minPts} \geq p + 1$, quindi avendo `cluster_scaled` 5 variabili, in questo caso possiamo scegliere `minPts=6`

Per scegliere `eps` si calcola la distanza dal suo `k`-esimo vicino, si ordinano queste distanze e si cerca un gomito, dove $k = \text{minPts} - 1$

```
library(dbSCAN)

kNNdistplot(cluster_scaled, k = 5)
abline(h = 0.8, col = "red", lty = 2)

db <- dbSCAN(cluster_scaled,
             eps = 0.8,
```

```

        minPts = 6)

table(db$cluster)
# 0 è il rumore

plot(cluster_data$Income,
      cluster_data$Total_spending,
      col = db$cluster + 1,
      pch = 19,
      xlab = "Income",
      ylab = "Total spending",
      main = "DBSCAN clustering")

legend("topright",
      legend = sort(unique(db$cluster)),
      col = sort(unique(db$cluster)) + 1,
      pch = 19,
      title = "Cluster")

plot(pca$x[, 1],
      pca$x[, 2],
      col = db$cluster + 1,
      pch = 19,
      xlab = "PC1",
      ylab = "PC2",
      main = "DBSCAN su PCA")

```

DBSCAN può individuare osservazioni rumorose, cioè clienti anomali rispetto alla densità generale dei dati. A differenza di K-means non richiede di fissare a priori il numero di cluster.

- DBSCAN individua clienti anomali? Quali vantaggi ha rispetto a K-means?

```

outliers <- data[db$cluster == 0, ]
summary(outliers)

```

PCA

- Applicare l'analisi delle componenti principali (PCA) ai dati standardizzati;

PCA cerca di ridurre il numero di variabili mantenendo la maggior parte dell'informazione contenute nei dati; le componenti principali sono combinazioni lineari delle variabili originali con determinati coefficienti (loadings)

La prima componente è la direzione con massima variabilità, poi la seconda cerca la massima variabilità residua rimanendo ortogonale

```

pca <- prcomp(cluster_scaled)

summary(pca)

```

Con sole due componenti abbiamo $57+19=76\%$ di variabilità spiegata.

- Si produca un grafico contenente la fedeltà della proiezione per numero di componenti e le osservazioni sulle prime due componenti;

Per rappresentare la varianza spiegata dalle componenti:

```
plot(pca,
     type = "l",
     main = "Scree plot")
```

Lo scree plot permette di identificare quante componenti spiegano gran parte della variabilità del dataset (in questo caso, basterebbero le prime due):

```
plot(pca$x[,1],
     pca$x[,2],
     pch=19,
     col="steelblue",
     xlab="PC1",
     ylab="PC2",
     main="Prime due componenti")
```

- Rappresentare i cluster ottenuti tramite K-means sulle prime due componenti: emergono gruppi chiaramente separati?

```
plot(pca$x[,1],
     pca$x[,2],
     col=km$cluster,
     pch=19,
     xlab="PC1",
     ylab="PC2",
     main="K-means sulle prime due componenti")

legend("topright",
      legend=paste("Cluster",1:2),
      col=1:2,
      pch=19)
```

La PCA non viene utilizzata qui per creare i cluster, ma per visualizzarli in uno spazio bidimensionale.; in particolare, se i cluster risultano ben separati sulle prime componenti, vuol dire che esistono differenze significative tra i gruppi e che tali differenze vengono calcolate da queste componenti.

Confronto tra algoritmi

- Confrontare:
 - clustering gerarchico
 - K-means
 - PAM
 - DBSCAN

utilizzando silhouette, dimensione dei cluster, interpretabilità e sensibilità agli outlier.

```
table(Ward = ward_clusters)
table(Kmeans = km$cluster)
table(PAM = pam_fit$clustering)
table(DBSCAN = db$cluster)
```

```
sil_ward <- silhouette(ward_clusters, dist_euclidean)
mean(sil_ward[, 3])
```

```
sil_km <- silhouette(km$cluster, dist_euclidean)
```

```

mean(sil_km[, 3])

sil_pam <- silhouette(pam_fit$clustering, dist_euclidean)
mean(sil_pam[, 3])

non_noise <- db$cluster != 0

if (length(unique(db$cluster[non_noise])) > 1) {
  sil_db <- silhouette(db$cluster[non_noise],
                      dist(cluster_scaled[non_noise, ]))
  mean(sil_db[, 3])
}

comparison <- data.frame(
  Metodo = c("Ward", "K-means", "PAM"),
  Silhouette_media = c(
    mean(sil_ward[, 3]),
    mean(sil_km[, 3]),
    mean(sil_pam[, 3])
  )
)

comparison

```

- Quale algoritmo sembra essere più adatto per questo dataset?
1. K-means è semplice e veloce, ma sensibile agli outlier e necessita di scegliere il numero di cluster
 2. PAM è più robusto perché utilizza osservazioni reali come rappresentanti dei cluster
 3. Ward produce una struttura gerarchica utile per visualizzare le fusioni tra gruppi (e può guidare nella scelta del numero di cluster per K-means ad esempio)
 4. DBSCAN è utile quando si vogliono identificare clienti anomali e cluster basati sulla densità, ma richiede di scegliere `eps` e `minPts`